

→ LR(0) :

• L → left to Right

The input is scanned from:
left → Right

• R → Rightmost derivation in reverse
LR parsing constructs the:
right most derivation in reverse

0 → zero lookahead

~~→ Why do we need LR(0)?~~

~~In simple shift-reduce parsing, we have to decide:~~

~~shift~~

~~or~~

~~reduce~~

~~This can create conflict!~~

→ LR(0) item:

A LR(0) item is a production with a **dot (.)** showing how much of the RHS has been recognized.

Consider:

$A \rightarrow XYZ$

If LR(0) items are:

$$A \rightarrow \cdot XYZ$$
$$A \rightarrow X \cdot YZ$$
$$A \rightarrow XY \cdot Z$$
$$A \rightarrow XYZ \cdot$$

The dot tells ^{us} the current position of the parser.

→ Understanding the dot:

Suppose:

$$A \rightarrow \cdot XYZ$$

meaning:

Nothing has been recognized yet.

After recognizing X:

$$A \rightarrow X \cdot YZ$$

meaning:

X has been recognized; now we expect y.

After recognizing XY:

$$A \rightarrow XY \cdot Z$$

meaning:

XY has been recognized; now we expect z.

→ Dots at the end:

$$A \rightarrow XYZ.$$

meaning:

The entire RHS has been recognized.

This indicates that a **reduction** can be performed.

→ Augmented Grammar:

Before constructing LR(0) items, we **augment** the grammar.

Suppose original grammar is:

$$S \rightarrow CC$$

$$C \rightarrow cC$$

$$C \rightarrow d$$

~~we~~ create a new start symbol:

S'

and add:

$$S' \rightarrow S$$

So the augmented grammar becomes:

$s' \rightarrow s$

$s \rightarrow cc$

$c \rightarrow cc$

$c \rightarrow d$

→ Why augment?

It gives a unique condition for acceptance.

When we reach:

$s' \rightarrow s.$

we know the input has been completely parsed.

$\therefore s' \rightarrow s. \Rightarrow \text{Accept}$

→ Initial LR(0) Item:

Start with:

$$S' \rightarrow \cdot S$$

Now apply closure.

Since the dot is before S, we need all production of S.

we have:

$$S \rightarrow \cdot CC$$

So add:

~~$$S \rightarrow \cdot CC$$~~

Now dot is before C.

we need all product of C:

$$C \rightarrow \cdot CC$$

$$C \rightarrow \cdot d$$

So the initial state is :

To :

$S' \rightarrow \cdot S$

$S \rightarrow \cdot CC$

$C \rightarrow \cdot CC$

$C \rightarrow \cdot d$

This is our first LR(0) state.

→ Closure :

Closure means :

If the dot is immediately before a non-terminal
add that non-terminal's production with the
dot at the beginning.

Eg ;

$A \rightarrow \alpha \cdot B \beta$

here dot is before B

∴ add

~~$B \rightarrow \cdot Y$~~

for every production :

$B \rightarrow Y$

Eg: Suppose:

$$S \rightarrow \cdot AB$$

$$A \rightarrow a$$

$$A \rightarrow b$$

Since dot is before A, add:

$$A \rightarrow \cdot a$$

$$A \rightarrow \cdot b$$

So closure becomes:

$$S \rightarrow \cdot AB$$

$$A \rightarrow \cdot a$$

$$A \rightarrow \cdot b$$

→ GOTO:

GOTO tell us:

What state do we reach after recognizing a particular grammar symbol?

Suppose we have:

$$A \rightarrow \alpha \cdot X \beta$$

If we move across X, the dot moves one position:

$$A \rightarrow \alpha X \cdot \beta$$

Then we calculate closure

So,

$$GOTO(I, X) = \text{closure (move dot over X)}$$

Eg of GOTO

Suppose Q_0 contains:

$$A \rightarrow \cdot XYZ$$

GOTO on X moves the dot:

$$A \rightarrow X \cdot YZ$$

So

$$I_0 \rightarrow X \rightarrow I_1$$

where:

I_1 contains

$$A \rightarrow X \cdot YZ$$

LR(0) Example:

Construct LR(0) Gfs for:

$$S \rightarrow CC$$

$$C \rightarrow cC$$

$$C \rightarrow d$$

⇒ Step 1:

Augment:

$$S' \rightarrow S$$

$$S \rightarrow CC$$

$$C \rightarrow cC$$

$$C \rightarrow d$$

State I_0 :

Starts with:

$$S' \rightarrow \cdot S$$

Closure given:

I_0 :

$$S' \rightarrow \cdot S$$

$$S \rightarrow \cdot CC$$

$$C \rightarrow \cdot cC$$

$$C \rightarrow \cdot d$$

GOTO from I_0 :

$S' \rightarrow \cdot s \rightarrow S$

$S \rightarrow \cdot cc \rightarrow C$

$C \rightarrow \cdot cC \rightarrow c$

$C \rightarrow \cdot d \rightarrow d$

So we need:

$GOTO(I_0, S)$

$GOTO(I_0, C)$

$GOTO(I_0, c)$

$GOTO(I_0, d)$

$I_1 = GOTO(I_0, s)$

Move dot across s :

$S' \rightarrow s \cdot$

$\therefore I_1 :$

$S' \rightarrow s \cdot$

This is accepting state.

$$I_2 = \text{GOTO}(I_0, C)$$

from :

$$S \rightarrow \cdot c c$$

move dot ^{over} first c:

$$S \rightarrow c \cdot c$$

So:

I_2 :

$$S \rightarrow c \cdot c$$

$$C \rightarrow \cdot c c$$

$$C \rightarrow \cdot d$$

Why did we add $C \rightarrow \cdot c c$ and $C \rightarrow \cdot d$?

Because after moving the dot, we have:

$$S \rightarrow c \cdot c$$

↑

The dot is before non-terminal c, so we apply closure.

$$I_3 = \text{GOTO}(I_0, c)$$

from

$$C \rightarrow \cdot c c$$

move dot:

~~Since~~ $C \rightarrow c.C$
 Since dot is before c , and c 's productions:

$I_3 :$

$C \rightarrow c.C$

$C \rightarrow .cC$

$C \rightarrow .d$

Similarly for all GOTO.

→ Canonical collection: v
 LR(0) items

$I_0 :$

$S' \rightarrow .S$

$S \rightarrow .cC$

$C \rightarrow .cC$

$C \rightarrow .d$

{ $\cdot$ के बारे में S और S का closure }

$I_1 :$

$S' \rightarrow S.$

{ Accept }

$I_2 :$ $S \rightarrow C.C$

$C \rightarrow .cC$

$C \rightarrow .d$

{ $\cdot$ के बारे में C और C का closure }

$I_3 :$ $C \rightarrow c.C$

$C \rightarrow .cC$

$C \rightarrow .d$

{ Same }

$$I_4: C \rightarrow d.$$

$$I_5: \underline{s \rightarrow c c.}$$

~~scribbled out text~~

$$I_6: C \rightarrow c c.$$

→ CYTO transition is

$$I_0 \rightarrow s \rightarrow I_1$$

$$I_0 \rightarrow C \rightarrow I_2$$

$$I_0 \rightarrow c \rightarrow I_3$$

$$I_0 \rightarrow d \rightarrow I_4$$

$$I_2 \rightarrow C \rightarrow I_5$$

$$I_2 \rightarrow c \rightarrow I_3$$

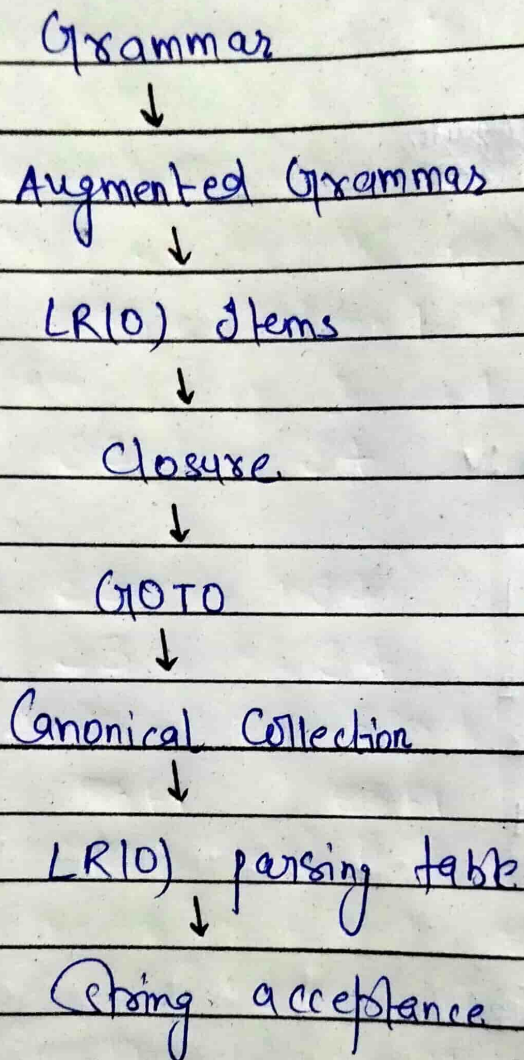
$$I_2 \rightarrow d \rightarrow I_4$$

$$I_3 \rightarrow C \rightarrow I_6$$

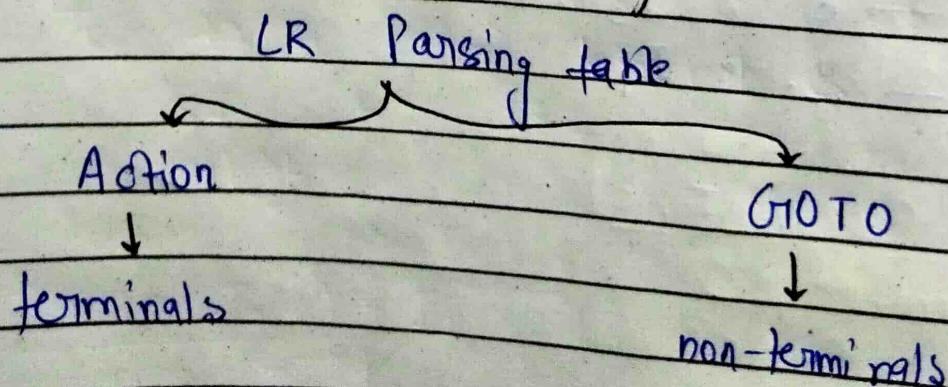
$$I_3 \rightarrow c \rightarrow I_3$$

$$I_3 \rightarrow d \rightarrow I_4$$

→ LR(0) parsing table :-



→ LR(0) table has two parts :-



→ Action:

Used when the next input symbol is a terminal.

Possible Actions:

Shift

Reduce

Accept

Error

→ GOTO:

Used when the symbol is a non-terminal.

→ Suppose: we will use the same grammar from the previous eg:

$S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$

⇒ Step 1: Augment the Grammar

Step 2: Construct LR(0) States

Step 2: GOTO Transition.

Step 4: LR(0) parsing table

~~States~~ ~~Action a~~ ~~Action d~~ ~~Action \$~~ ~~GOTO S~~ ~~GOTO C~~

LR(0)

Always
Start →
from here
first
complete
zero
then
1, 2, ...

States	Action c	Action d	Action \$	GOTO S	GOTO C
0	S3	S4		1	2
1			ACCEPT		
2	S3	S4			5
3	S3	S4			6
4	R3	R3	R3		
5	R1	R1	R1		
6	R2	R2	R2		

→ Fill shift entries: ~ {Action part}
 {dot before terminal}
 • $c \rightarrow \cdot cc$ {Using GOTO transition}

The dot is before terminal c.

∴ shift c

$$\text{GOTO}(I_0, c) = I_3$$

$$\therefore \text{Action}[0, c] = S3$$

• shift Entry means dot before terminals

{Just same as we do in shift-Reduce?}

• $C \rightarrow \cdot d$

The dot is before terminal d.

$\therefore$ shift d

$GOTO(I_0, d) = I_4$

$\therefore Action(0, d) = s_4$

→ Fill GOTO Entries :-

If the transition is on non-terminals, put it in the GOTO section.

{dot before non-terminals}

$GOTO(I_0, S) \rightarrow I_1$

So:

$GOTO[0, S] = 1$

$GOTO(0, c) = I_2$

So :

$GOTO[0, c] = 2$

→ Fill reduce entries :-

Suppose the state contains :

$C \rightarrow d.$

The dot is at the end.

{ dot is at the end / for both terminal / non-terminal }

this state means :

Reduce by $C \rightarrow d$

In LR(0) because there is no lookahead, the reduction is placed under all terminal columns.

Reason:

Our terminals are:

$c, d, \epsilon, \$$

∴ State I_4 gets:

R3 R3 R3

where production 3 is

$C \rightarrow d$

The production is complete so reduce by $C \rightarrow d$ but it does not check whether reduction is appr. for the next input symbol

State I_6 :

$C \rightarrow cC$.

$\therefore$ Reduce Reduce by $C \rightarrow cC$

This is production (2).

So

R_2

is placed under all terminal columns.

— Acceptance Entry:

State I_1 contains:

$S' \rightarrow S$.

This is the augmented production with dot at the end.

$\therefore$ Action $[1, \$] = \text{Accept}$

Only $\$$ gets Accepts.

We are study this topic first time so we find all four condition that can happen separately from the next question we start from $0(I_0)$ and find all then go to next $I_1, I_2, \dots$

Now here, find rest of them:

- I_0 completed.
- I_1 completed

• I_2 :

$$S \rightarrow C \cdot C$$

dot before non-terminal

$$\text{GOTO}[I_2, C] = I_5$$

$$\text{GOTO}[I_2, c] = 5$$

$$C \rightarrow \cdot C C$$

dot before terminal

shift entry

$$\text{GOTO}[I_2, c] = I_3$$

$$\text{Action}[I_2, c] = S3$$

$C \rightarrow \cdot d$

Action $[I_2, d] = S_4$

$I_3 :$

$C \rightarrow c \cdot C$

dot before non-terminal

GOTO $[I_3, c] = 6$

$C \rightarrow \cdot c C$

dot before terminal

Action $[I_3, c] = S_3$

$C \rightarrow \cdot d$

Action $[I_3, d] = S_4$

$I_4 :$ completed

$I_5 : S \rightarrow c c \cdot$

dot at end

reduce entry

$S \rightarrow c c \rightarrow$ production (1)

Note: $S \rightarrow CC$
 $C \rightarrow Cc/d$

here, Production (1) $\rightarrow S \rightarrow CC$
" (2) $\rightarrow C \rightarrow Cc$
" (3) $\rightarrow C \rightarrow d$.

Initial			
Date			

Fill Row in all column of S .

I_0 : Completed

Hence Grammar is LR(0).

Conflicts :-

Suppose one table cell gets:

Shift

+

Reduce

$\Rightarrow$ Shift-Reduce conflict

Or:

Reduce/Reduce

$\Rightarrow$ Reduce-Reduce conflict

If LR(0) table has any conflict:

Grammar is not LR(0)

Akash